PSDL Lab assignment 6 to 12

information technology (Zeal College of Engineering and Research)

Group B: Assignment No 6

Problem Statement:

Write an Embedded C program to interface PIC 18FXXX with LED & blinking it using specified delay.

Objective:
To interface PIC 18F4550 with LED & blinking it using specified delay.

Outcome:
LED will be blinking with delay specified.

CO Relevance: CO3

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

1. Connect the LED to a suitable pin of the PIC 18F4550 microcontroller. Make sure to include a current-limiting resistor to prevent excessive current flow.
2. Configure the pin connected to the LED as an output pin in the microcontroller's TRIS register. Set the corresponding bit to 0 to configure it as an output.
3. Write a loop that toggles the state of the output pin to turn the LED on and off. Use the LAT register to set or clear the pin.
4. Specify a delay of 100 milliseconds between each state change of the LED using a delay function.
5. Repeat the loop indefinitely to maintain the continuous blinking of the LED.

Program:

```
#include<pic18f4550.h>
void delay()
{
        unsigned int i;
        for(i=0;i<30000;i++);
}

void main()
{
   unsigned char i, key = 0;
   TRISB = 0x00;                    //LED pins as output
   LATB = 0x00;
   ADCON1 = 0x0F;                   //set pins as Digital
```

```
   TRISAbits.TRISA2 = 1;              //set RA2 as input
   TRISAbits.TRISA3 = 1;              //set RA3 as input

   TRISAbits.TRISA5 = 0;              //set buzzer pin RA5 as output
   TRISAbits.TRISA4 = 0;              //set relay pin RA4 as output
   while(1)
   {
      //LATAbits.LA2 = 1;
      //LATAbits.LA3 = 1;
      if(PORTAbits.RA2 == 0) key =0;     //If button1 pressed
      if(PORTAbits.RA3 == 0) key =1;     //If button2 pressed

      if(key == 0)
      {
         LATAbits.LATA4 = 1;          //Relay OFF
         LATAbits.LATA5 = 0;          //Buzzer OFF
         for(i=0;i<8;i++)             //Chase LED right to left
         {
            LATB = 1<<i;
            delay();
            LATB = 0x00;
            delay(100);
         }
      }
      if(key == 1)
      {
         LATAbits.LATA4 = 0;          //Relay ON
         LATAbits.LATA5 = 1;          //Buzzer ON
         for(i=7;i> 0;i--)            //Chase LED left to right
         {
            LATB = 1<<i;
            delay(100);
            LATB = 0x00;
            delay(100);
         }
      }

   }
}
```

Result:

Output can be checked on kit.

Conclusion:

The program of interface PIC 18F4550 with LED & blinking using delay is verified successfully on kit.

Group B: Assignment No 7

Problem Statement:

Write an Embedded C program for Timer programming ISR based buzzer on/off.

Objective:
To interface PIC 18F4550 for Timer programming ISR based buzzer on/off.

Outcome:
LED will go with delay and Buzzer will be on/off.

CO Relevance: CO3

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

1. Configure the Timer module of the PIC 18F4550 microcontroller. Select the appropriate Timer module and configure its settings such as prescaler, period, and interrupt enable bit.
2. Initialize the necessary registers for the Timer module. This may involve setting the prescaler, enabling the Timer interrupt, and setting the Timer period.
3. Write the ISR to handle the Timer interrupt. In the ISR, toggle the state of the pin connected to the buzzer to turn it on/off.
4. Enable global interrupts in the microcontroller's configuration registers.

Program:

```
#include <pic18f4550.h>
#include <stdint.h>
#include <xc.h>
#define _XTAL_FREQ 8000000

#define BUZZER_PIN PORTDbits.RD4

volatile uint8_t timer_count = 0;
volatile uint8_t buzzer_state = 0;

void interrupt_ISR() {
   if (TMR1IF) {
      TMR1IF = 0;
      timer_count++;
      if (timer_count >= 50) {
```

```
            timer_count = 0;
            buzzer_state = !buzzer_state;
            if (buzzer_state) {
               BUZZER_PIN = 1;
            } else {
               BUZZER_PIN = 0;
            }
         }
      }
   }

void main(void) {
   // Set Buzzer pin as output
   TRISDbits.TRISD4 = 0;

   // Set Timer1 to CTC mode, prescaler 256, and interrupt every 1ms
   T1CONbits.TMR1ON = 0;
   T1CONbits.T1CKPS = 3;
   T1CONbits.T1SYNC = 0;
   T1CONbits.TMR1CS = 0;
   T1CONbits.T1OSCEN = 0;
   T1CONbits.T1RD16 = 0;
   TMR1H = 0xFC;
   TMR1L = 0x18;
   TMR1IF = 0;
   TMR1IE = 1;
   PEIE = 1;
   GIE = 1;
   T1CONbits.TMR1ON = 1;
   while (1)
   {
      // Make the buzzer beep continuously with a different frequency
      BUZZER_PIN = 1;
      __delay_ms(100);
      BUZZER_PIN = 0;
      __delay_ms(1000);
   }
}
```

Result:

Output can be checked on kit.

Conclusion:

The program to interface PIC 18F4550 for Timer programming ISR based buzzer on/off.

.

Group B: Assignment No 8

Problem Statement:

Write an Embedded C program for External interrupt input switch press, output at relay.

Objective:
To write an Embedded C program for External interrupt input switch press, output at relay.

Outcome:
To implement External interrupt input switch press, output at relay.

CO Relevance: CO3

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

1. Connect the switch to an external interrupt pin of the PIC 18F4550 microcontroller. Ensure that the switch is properly debounced to eliminate false triggering.
2. Configure the external interrupt pin as an input in the microcontroller's TRIS register.
3. Enable the external interrupt module and configure its settings, such as interrupt edge detection (rising edge, falling edge, or both) and priority.
4. Write an interrupt service routine (ISR) to handle the external interrupt. In the ISR, check the state of the switch and control the output relay accordingly.
5. Configure the pin connected to the relay as an output in the microcontroller's TRIS register.

Program:

```
#include <pic18f4550.h>

#define BUZZER_PIN LATAbits.LATA5

void interrupt_extint_isr(void)
{
   unsigned int i;
   if(INT1F)
   {
      INT1F = 0;
      INT1IE = 0;
      BUZZER_PIN = ~BUZZER_PIN;
      for(i=0; i<10000; i++);      //small delay for debouncing
      INT1IE = 1;
   }
```

```
}

int main()
{
   ADCON1 = 0x0F;          //set pins as Digital
   TRISAbits.TRISA5 = 0;   //set buzzer pin RA5 as output
   TRISBbits.TRISB1 = 1;   //Interrupt pin as input
   BUZZER_PIN = 1;

   INT1IE  =  1;                 //Enable external interrupt INT1
   INTEDG1 =  0;                   //Interrupt on falling edge
   GIE     =  1;                 // Enable global interrupt

   while(1);
}
```

Result:

Output can be checked on kit.

Conclusion:

The program of interface PIC 18F4550 for external interrupt input switch press, output at relay

Group B: Assignment No 9

Problem Statement:

Write an Embedded C program for LCD interfacing with PIC 18FXXX.

Objective:
Write an Embedded C program for LCD interfacing with PIC 18F4550.

Outcome:
To implement LCD interfacing with PIC 18F4550.

CO Relevance: CO3

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

1. Connect the LCD to the appropriate pins of the PIC 18F4550 microcontroller. The LCD typically requires connections for data lines (D0-D7), control lines (RS, RW, and EN), and possibly additional control lines for backlight control.
2. Configure the data and control pins as output pins in the microcontroller's TRIS register.
3. Initialize the LCD by sending specific commands and data sequences. This typically involves sending initialization commands such as function set, display on/off control, entry mode set, etc.
4. Write functions or routines to send commands and data to the LCD. These functions will set the control lines (RS, RW, and EN) to specify whether the data being sent is a command or data and then send the appropriate information over the data lines.
5. Use the LCD functions to display desired text or information on the LCD. This can include sending commands to position the cursor, clear the display, or print characters on specific lines.

Program:

```
#include <pic18f4550.h>

#define LCD_EN LATAbits.LA1
#define LCD_RS LATAbits.LA0
#define LCDPORT LATB


void lcd_delay(unsigned int time)
{
 unsigned int i , j ;
```

```
   for(i = 0; i < time; i++)
   {
        for(j=0;j<100;j++);
   }
}


void SendInstruction(unsigned char command)
{
    LCD_RS = 0;              // RS low : Instruction
    LCDPORT = command;
    LCD_EN = 1;              // EN High
    lcd_delay(10);
    LCD_EN = 0;              // EN Low; command sampled at EN falling edge
    lcd_delay(10);
}

void SendData(unsigned char lcddata)
{
    LCD_RS = 1;              // RS HIGH : DATA
    LCDPORT = lcddata;
    LCD_EN = 1;              // EN High
    lcd_delay(10);
    LCD_EN = 0;              // EN Low; data sampled at EN falling edge
    lcd_delay(10);
}

void InitLCD(void)
{
   ADCON1 = 0x0F;
   TRISB = 0x00; //set data port as output
   TRISAbits.RA0 = 0; //RS pin
   TRISAbits.RA1 = 0; // EN pin

   SendInstruction(0x38);     //8 bit mode, 2 line,5x7 dots
   SendInstruction(0x06);      // entry mode
   SendInstruction(0x0C);      //Display ON cursor OFF
   SendInstruction(0x01);     //Clear display
   SendInstruction(0x80);     //set address to 1st line

}
/*****************************************************************************************
*********************************/

unsigned char *String1 = " WELCOME TO";
```

```
unsigned char *String2 = " PSDL";

void main(void)
{
   ADCON1 = 0x0F;
   TRISB = 0x00;       //set data port as output
   TRISAbits.RA0 = 0;  //RS pin
   TRISAbits.RA1 = 0;  // EN pin

   SendInstruction(0x38);      //8 bit mode, 2 line,5x7 dots
   SendInstruction(0x06);      // entry mode
   SendInstruction(0x0C);      //Display ON cursor OFF
   SendInstruction(0x01);      //Clear display
   SendInstruction(0x80);      //set address to 1st line

 while(*String1)
 {
  SendData(*String1);
  String1++;
 }

 SendInstruction(0xC0);      //set address to 2nd line
 while(*String2)
 {
  SendData(*String2);
  String2++;
 }

 while(1);

}
```

Result:

Output can be checked on kit.

Conclusion:

The program of interface PIC 18F4550 for LCD interfacing is verified successfully.

Group C: Assignment No 10

Problem Statement:

Write an Embedded C program for Generating PWM signal for servo motor/DC motor.

Objective:
Write an Embedded C program for Generating PWM signal for servo motor/DC motor.

Outcome:
To implement PWM signal for servo motor/DC motor.

CO Relevance: CO3

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

1. Configure the PWM module of the PIC 18F4550 microcontroller. Select the appropriate PWM module and configure its settings such as frequency, duty cycle, and resolution.
2. Initialize the necessary registers for the PWM module. This may involve setting the PWM period, selecting the PWM output pin, and configuring the PWM duty cycle.
3. Write the code to control the servo motor or DC motor using the PWM module. For a servo motor, you will typically set the duty cycle based on the desired angle of rotation. For a DC motor, you will typically set the duty cycle based on the desired speed or direction of rotation.
4. Compile and upload the code to the microcontroller.

Program:

```
/*      Calculations
 * Fosc = 48MHz
 *
 * PWM Period = [(PR2) + 1] * 4 * TMR2 Prescale Value / Fosc
 * PWM Period = 200us
 * TMR2 Prescale = 16
 * Hence, PR2 =  149 or 0x95
 *
 * Duty Cycle = 10% of 200us
 * Duty Cycle = 20us
 * Duty Cycle = (CCPR1L:CCP1CON<5:4>) * TMR2 Prescale Value / Fosc
 * CCP1CON<5:4> = <1:1>
 * Hence, CCPR1L = 15 or 0x0F
```

```
 */


#include <pic18f4550.h>
typedef unsigned char bit;


unsigned char count=0;
bit TIMER,SPEED_UP;

void timer2Init(void)
{
   T2CON   =   0b00000010;          //Prescalar = 16; Timer2 OFF
   PR2     =   0x95;                //Period Register
}

/*void Interrupt_Init(void)
{
   INT1IE  =   1;                   //Enable external interrupt INT1
   INTEDG1 =   0;                   //Interrupt on falling edge
   GIE     =   1;                  // Enable global interrupt
}*/

/*void interrupt ISR(void)
{
    if (INT1IF)                     // If the external interrupt flag is 1, do .....
   {
      INT1IF  =   0;                // Reset the external interrupt flag
      if(SPEED_UP)
      {
         if(count < 8)
         {
            count++;
            CCPR1L = (count * 0x0F);      //Increment duty cycle
         }

         else SPEED_UP = 0;

      }

      else
      {
         if(count > 1)
         {
            count--;
```

```
            CCPR1L = (count * 0x0F);     //Decrement duty cycle
          }
          else SPEED_UP = 1;
       }
    }
}*/
void delay(unsigned int time)
{
   unsigned int i,j;
   for(i=0;i<time;i++)
      for(j=0;j<1000;j++);
}
void main(void)
{
   unsigned int i;
   TRISCbits.TRISC1    = 0;          //RC1 pin as output
   TRISCbits.TRISC2    = 0;          //CCP1 pin as output
   LATCbits.LATC1      = 0;
   CCP1CON  =  0b00111100;          //Select PWM mode; Duty cycle LSB CCP1CON<4:5> = <1:1>
   CCPR1L   =  0x0F;                //Duty cycle 10%
   timer2Init();                   //Initialise Timer2
   //Interrupt_Init();               //Initialise interrupts
   //SPEED_UP = 1;
   TMR2ON = 1;                     //Timer2 ON

   while(1)                        //Loop forever
   {
      for(i=15;i<150;i++)
      {
         CCPR1L = i;
         delay(100);
      }
      for(i=150;i>15;i--)
      {
         CCPR1L = i;
         delay(100);
      }
   }
}
```

Result:

Output can be checked on kit.

Conclusion:

The program of interface PIC 18F4550 for Embedded C program for Generating PWM signal for servo motor/DC motor.

Group C: Assignment No 11

Problem Statement:

Write an Embedded C program for Temperature sensor interfacing using ADC & display on LCD.

Objective:
Write an Embedded C program for Temperature sensor interfacing using ADC & display on LCD.

Outcome:
Measuring the temperature.

CO Relevance: CO3

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

1. Connect the temperature sensor to an analog input pin of the microcontroller. Ensure that the sensor is properly powered and grounded.
2. Configure the ADC module of the microcontroller. Select the appropriate ADC module and configure its settings such as reference voltage, resolution, and acquisition time.
3. Initialize the necessary registers for the ADC module. This may involve setting the ADC reference voltage, selecting the ADC input channel, and configuring the ADC result format.
4. Write a function to read the temperature from the ADC. This function will start the ADC conversion, wait for the conversion to complete, and retrieve the converted digital value.
5. Calculate the temperature value based on the ADC reading using the appropriate formula or lookup table. This calculation will depend on the specific temperature sensor being used.
6. Configure the LCD module of the microcontroller. Select the appropriate LCD module and configure its settings such as display mode, cursor position, and data transmission mode.
7. Write functions or routines to send commands and data to the LCD. These functions will set the control lines (RS, RW, and EN) to specify whether the data being sent is a command or data and then send the appropriate information over the data lines.
8. Use the LCD functions to display the temperature on the LCD. This may involve sending commands to position the cursor and print the temperature value as characters on the display.

Program:

```
#include <pic18f4550.h>
#include <stdio.h>
#define LCD_EN LATAbits.LA1
#define LCD_RS LATAbits.LA0
#define LCDPORT LATB

unsigned char str[16];

void lcd_delay(unsigned int time)
{
 unsigned int i , j ;

   for(i = 0; i < time; i++)
   {
        for(j=0;j<100;j++);
   }
}


void SendInstruction(unsigned char command)
{
    LCD_RS = 0;             // RS low : Instruction
    LCDPORT = command;
    LCD_EN = 1;             // EN High
    lcd_delay(10);
    LCD_EN = 0;             // EN Low; command sampled at EN falling edge
    lcd_delay(10);
}

void SendData(unsigned char lcddata)
{
    LCD_RS = 1;             // RS HIGH : DATA
    LCDPORT = lcddata;
    LCD_EN = 1;             // EN High
    lcd_delay(10);
    LCD_EN = 0;             // EN Low; data sampled at EN falling edge
    lcd_delay(10);
}

void InitLCD(void)
```

```
{
   ADCON1 = 0x0F;
   TRISB = 0x00; //set data port as output
   TRISAbits.RA0 = 0; //RS pin
   TRISAbits.RA1 = 0; // EN pin

   SendInstruction(0x38);      //8 bit mode, 2 line,5x7 dots
   SendInstruction(0x06);       //entry mode
   SendInstruction(0x0C);       //Display ON cursor OFF
   SendInstruction(0x01);      //Clear display
   SendInstruction(0x80);      //set address to 0
}

void LCD_display(unsigned int row, unsigned int pos, unsigned char *ch)
{
   if(row==1)
      SendInstruction(0x80 | (pos-1));
   else
      SendInstruction(0xC0 | (pos-1));

   while(*ch)
      SendData(*ch++);
}

void ADCInit(void)
{
   TRISEbits.RE1 = 1;              //ADC channel 6 input
   TRISEbits.RE2 = 1;              //ADC channel 7 input

   ADCON1 = 0b00000111;            //Ref voltages Vdd & Vss; AN0 - AN7 channels Analog
   ADCON2 = 0b10101110;            //Right justified; Acquisition time 4T; Conversion clock Fosc/64
}

unsigned short Read_ADC(unsigned char Ch)
{
   ADCON0 = 0b00000001 | (Ch<<2);      //ADC on; Select channel;
   GODONE = 1;             //Start Conversion

   while(GO_DONE == 1 );    //Wait till A/D conversion is complete
   return ADRES;                 //Return ADC result
}

int main(void)
{
   unsigned int temp;
```

```
    InitLCD();
    ADCInit();
    LCD_display(1,1,"Temperature:");
    while(1)
    {
       temp = Read_ADC(7);
       temp = ((temp * 500) / 1023);
       sprintf(str,"%d'C  ",temp);
       LCD_display(2,1,str);
       lcd_delay(5000);
    }
    return 0;
}
```

Result:

Output can be checked on kit.

Conclusion:

The program of interface PIC 18F4550 for Temperature sensor interfacing using ADC & display on LCD is done successfully.

Group D: Assignment No 12

Problem Statement:

Write a simple program using Open-source prototype platform like Raspberry-Pi/Beagle board/Arduino for digital read/write using LED and switch Analog read/write using sensor and actuators.

Objective:
Read/write using LED and switch Analog read/write using sensor and actuators.

Outcome:
LED will be on and off with read and write opearations.

CO Relevance: CO3

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

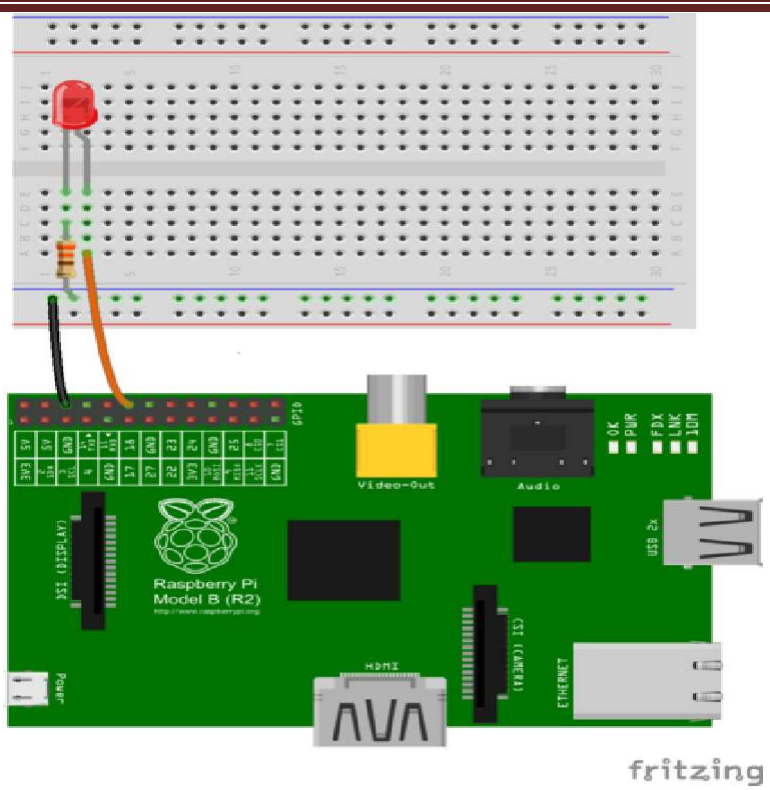
Theory Concepts:

An application of blinking LEDs using Raspberry Pi

In this assignment, we will learn how to interface an LED with Raspberry Pi. To implement this, we will gather the requirements and follow the instructions given below.

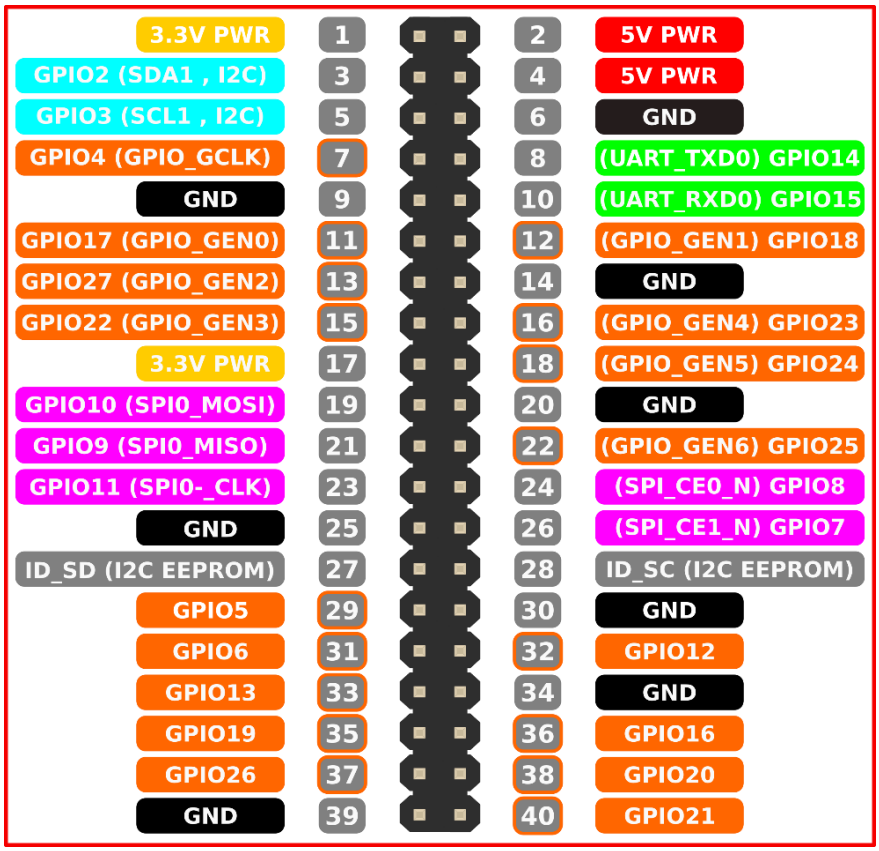
Requirements:

1. Raspberry Pi3 Kit
2. Breadboards
3. LEDs
4. Jumper wires



Instructions to Glow LED:

Use GPIO diagram of Raspberry Pi, which will give you understanding of GPIO structure.



Algorithm:

1. Connect GPIO 18 (Pin No. 12) of Raspberry Pi to the Anode of the LED through connecting jumper wires and Breadboard.
2. Connect Ground of Raspberry Pi to Cathode of the LED through connecting wires and breadboard.
3. Now power up your Raspberry Pi and boot.
4. Open terminal and type *nano blinkled.py*
5. It will open the nano editor. Use following pseudo code in the python to blink LED and save

```
import RPi.GPIO as GPIO import time
GPIO.setmode(GPIO.BCM)
GPIO.setup(18, GPIO.OUT)
while True:
    GPIO.output(18, GPIO.HIGH)
    time.sleep(1) GPIO.output(18,
    GPIO.LOW)
    time.sleep(1)
```

6. Now run the code. Type *python blinkled.py*
7. Now LED will be blinking at an interval of 1s. You can also change the interval by modifying time.sleep in the file.
8. Press Ctrl+C to stop LED from blinking.

Program:

```
import RPi.GPIO as GPIO
import spidev
import time

# Set up GPIO mode
GPIO.setmode(GPIO.BCM)

# Digital read/write using LED and switch
led_pin = 17
switch_pin = 18

GPIO.setup(led_pin, GPIO.OUT)
GPIO.setup(switch_pin, GPIO.IN, pull_up_down=GPIO.PUD_UP)

def toggle_led(pin):
   GPIO.output(pin, not GPIO.input(pin))

# Analog read using MCP3008 ADC
```

```
spi = spidev.SpiDev()
spi.open(0, 0)

def read_adc(channel):
    adc_cmd = [1, 8 + channel << 4, 0]
    adc_result = spi.xfer2(adc_cmd)
    adc_value = ((adc_result[1] & 3) << 8) + adc_result[2]
    return adc_value

try:
    while True:
        # Read switch state and toggle LED
        if GPIO.input(switch_pin) == GPIO.LOW:
            toggle_led(led_pin)
            time.sleep(0.2)  # Debounce delay

        # Read analog value from channel 0 of MCP3008
        analog_value = read_adc(0)
        print("Analog value: {}".format(analog_value))

        time.sleep(0.1)  # Delay between readings

except KeyboardInterrupt:
    GPIO.cleanup()
    spi.close()
```

Result:

Output can be checked on kit.

Conclusion:

The program of interface of Raspberry pi for read/write using LED and switch Analog read/write using sensor and actuators is done successfully.

****